Experiment-7

Aim: Demonstration of GNU/Linux IPC mechanisms- Pipes, Semaphores, Shared memory, Message Queues.

a) Pipes:

Description

An unnamed pipe is a temporary, one-way communication channel commonly used by related processes. It has a read end and a write end, but no file-system name. This program demonstrates basic pipe operations within a single process by writing three messages and reading them back in order.

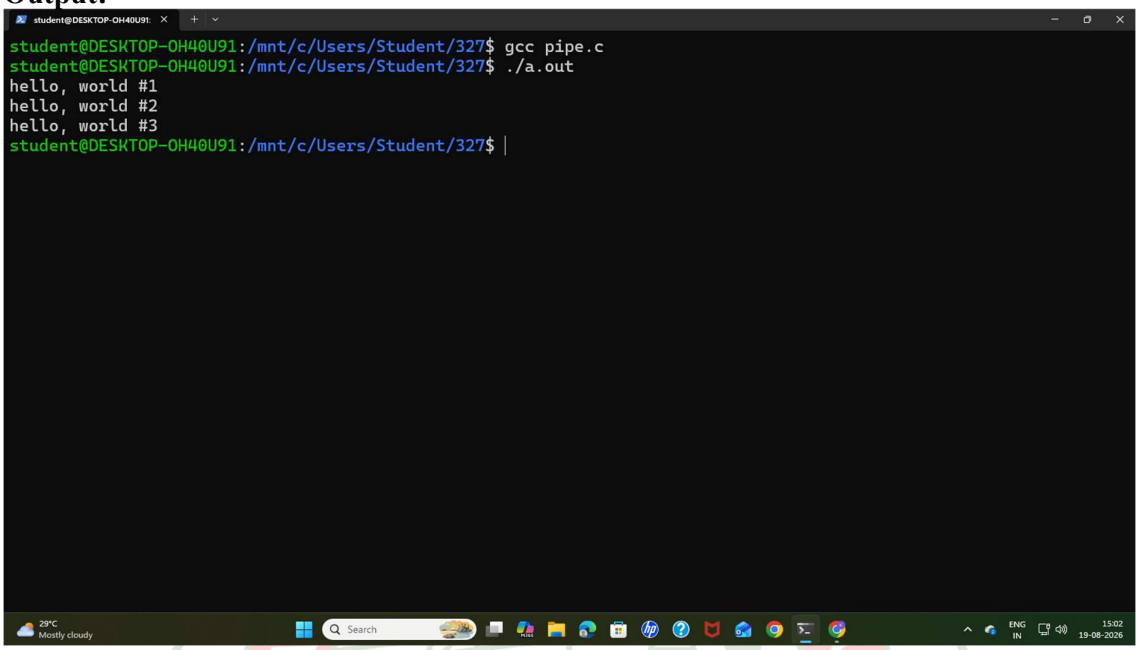
Algorithm

1. Start and declare three messages, a buffer and two pipe descriptors.
2. Create the pipe using pipe(); terminate if creation fails.
3. Write each fixed-size message to the write end of the pipe.
4. Read three messages from the read end, one at a time.
5. Display each message and terminate the program.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define MSGSIZE 16
char* msg1 = "hello, world #1";
char* msg2 = "hello, world #2";
char* msg3 = "hello, world #3";
int main(){
   char buf[MSGSIZE];
   int p[2], i;
   if (pipe(p) < 0)
      exit(1);
   write(p[1], msg1, MSGSIZE);
   write(p[1], msg2, MSGSIZE);
   write(p[1], msg3, MSGSIZE);
   for (i = 0; i < 3; i++) {
      read(p[0], buf, MSGSIZE);
      printf("%s\n", buf);
   }
   return 0;
}
```

Output:



b) Named Pipes.

Description

A named pipe, or FIFO, is a special file that allows unrelated processes to exchange data. The reader opens /tmp/myfifo, waits for a writer, receives a message and displays it. The FIFO must already exist before the reader opens it; the writer program creates it.

Algorithm

1. Start and set the FIFO path to /tmp/myfifo.
2. Open the FIFO in read-only mode; report an error and stop if opening fails.
3. Read the message into the buffer and display it.
4. Close the descriptor and remove the FIFO using unlink().
5. Terminate the program.

fifoReader Code:

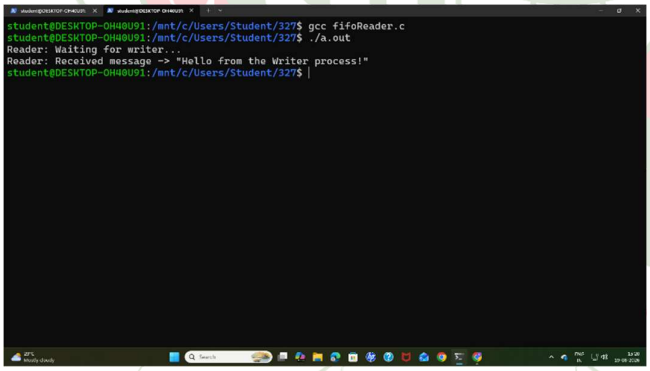
```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main() {
    char *myfifo = "/tmp/myfifo";
    char buffer[1024];
    printf("Reader: Waiting for writer...\n");
    int fd = open(myfifo, O_RDONLY);
    if (fd == -1) {
        perror("Failed to open pipe");
        return 1;
    }
    read(fd, buffer, sizeof(buffer));
    printf("Reader: Received message -> \"%s\"\n", buffer);
```

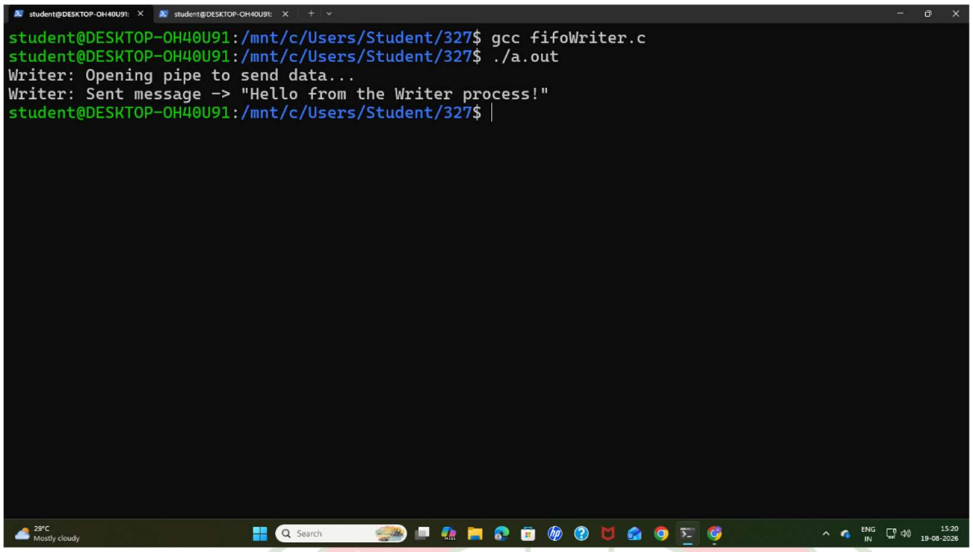
```
    close(fd);
    unlink(myfifo);
    return 0;
}
```

fifoWriter Code:

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
int main() {
    char *myfifo = "/tmp/myfifo";
    mkfifo(myfifo, 0666);
    printf("Writer: Opening pipe to send data...\n");
    int fd = open(myfifo, O_WRONLY);
    if (fd == -1) {
        perror("Failed to open pipe");
        return 1;
    }
    char message[] = "Hello from the Writer process!";
    write(fd, message, strlen(message) + 1);
    printf("Writer: Sent message -> \"%s\"\n", message);
    close(fd);
    return 0;
}
```

Output:





c) Demonstrate Named pipes for dynamic conversation from client and server

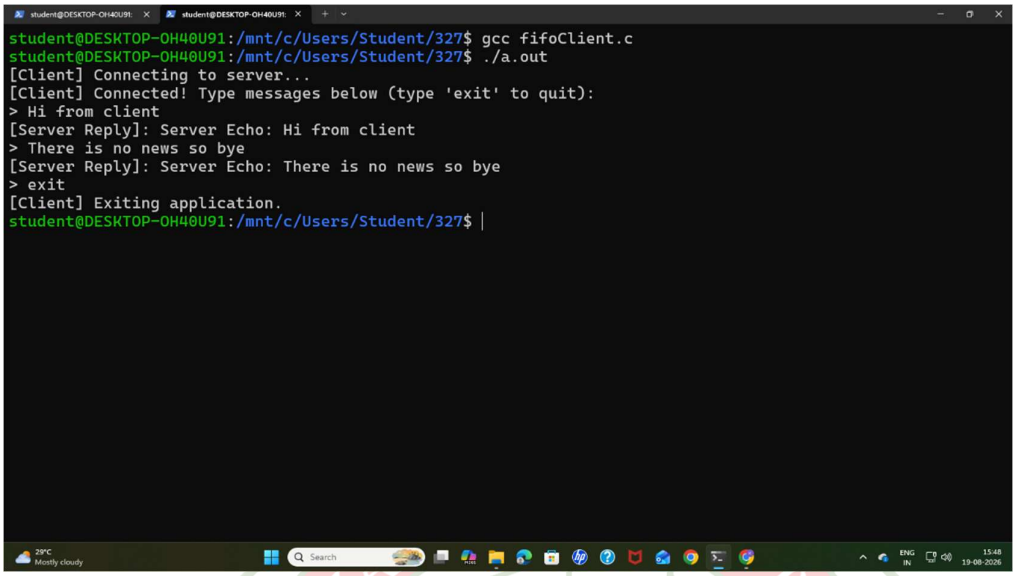
Client Code:

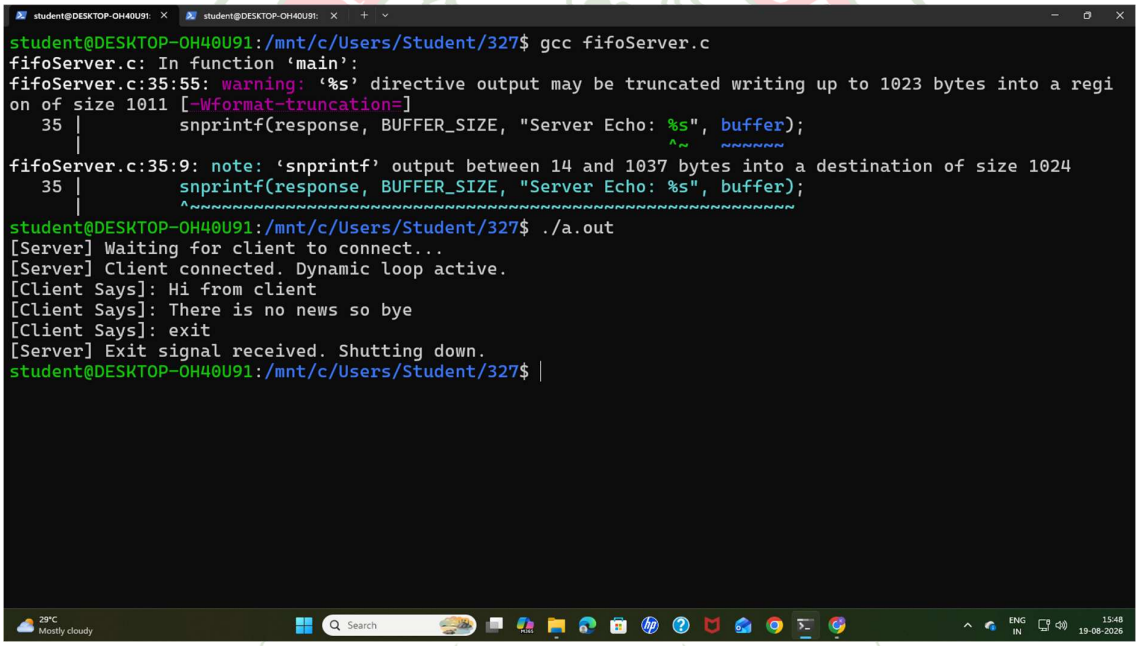
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
#define FIFO_C2S "/tmp/fifo_client_to_server"
#define FIFO_S2C "/tmp/fifo_server_to_client"
#define BUFFER_SIZE 1024
int main() {
    int fd_read, fd_write;
    char buffer[BUFFER_SIZE];
    mkfifo(FIFO_C2S, 0666);
    mkfifo(FIFO_S2C, 0666);
    printf("[Server] Waiting for client to connect...\n");
    fd_read = open(FIFO_C2S, O_RDONLY);
    fd_write = open(FIFO_S2C, O_WRONLY);
    printf("[Server] Client connected. Dynamic loop active.\n");
    while (1) {
        memset(buffer, 0, BUFFER_SIZE);
        if (read(fd_read, buffer, BUFFER_SIZE) <= 0) {
            printf("[Server] Client disconnected.\n");
            break;
        }
        printf("[Client Says]: %s", buffer);
        if (strncmp(buffer, "exit", 4) == 0) {
            printf("[Server] Exit signal received. Shutting down.\n");
            break;
        }
        char response[BUFFER_SIZE];
        snprintf(response, BUFFER_SIZE, "Server Echo: %s", buffer);
```

```
        write(fd_write, response, strlen(response) + 1);
    }
    close(fd_read);
    close(fd_write);
    unlink(FIFO_C2S);
    unlink(FIFO_S2C);
    return 0;
}
```

Server Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
#define FIFO_C2S "/tmp/fifo_client_to_server"
#define FIFO_S2C "/tmp/fifo_server_to_client"
#define BUFFER_SIZE 1024
int main() {
    int fd_write, fd_read;
    char buffer[BUFFER_SIZE];
    printf("[Client] Connecting to server...\n");
    fd_write = open(FIFO_C2S, O_WRONLY);
    fd_read = open(FIFO_S2C, O_RDONLY);
    printf("[Client] Connected! Type messages below (type 'exit' to quit):\n");
    while (1) {
        printf("> ");
        fflush(stdout);
        memset(buffer, 0, BUFFER_SIZE);
        if (fgets(buffer, BUFFER_SIZE, stdin) == NULL) break;
        write(fd_write, buffer, strlen(buffer) + 1);
        if (strncmp(buffer, "exit", 4) == 0) {
            printf("[Client] Exiting application.\n");
            break;
        }
        memset(buffer, 0, BUFFER_SIZE);
        if (read(fd_read, buffer, BUFFER_SIZE) > 0) {
            printf("[Server Reply]: %s", buffer);
        }
    }
    close(fd_write);
    close(fd_read);
    return 0;
}
```





d) Demonstrate Producer- consumer problem using the semaphores.

Description

This program uses producer and consumer threads with a five-element circular buffer. The semaphore, named mutex in the code, is initialized to zero and counts available items. The producer signals after storing an item, while the consumer waits before reading one .This

example produces exactly five items; it does not implement general buffer-capacity control or a separate mutual-exclusion lock.

Algorithm

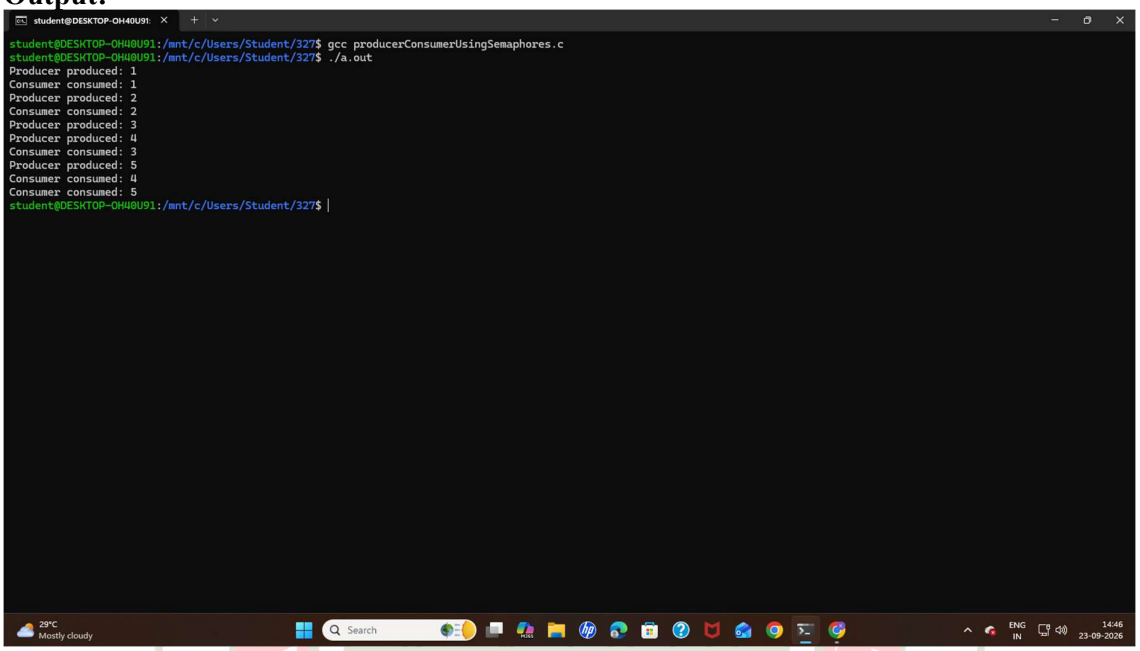
1. Start, set the circular-buffer indices to zero and initialize the semaphore to zero.
2. Create the producer and consumer threads.
3. Producer: store each value from 1 to 5, display it and advance the write index modulo SIZE.
4. After each item, post the semaphore and sleep for one second.
5. Consumer: wait on the semaphore, read and display an item, and advance the read index modulo SIZE. Repeat five times, sleeping for two seconds after each item.
6. Wait for both threads to finish, destroy the semaphore and terminate.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define SIZE 5
int buffer[SIZE];
int in = 0;
int out = 0;
sem_t mutex;
void* producer(void* arg){
   for (int i = 1; i <= 5; i++){
      buffer[in] = i;
      printf("Producer produced: %d\n", i);
      in = (in + 1) % SIZE;
      sem_post(&mutex);
      sleep(1);
   }
   return NULL;
}
void* consumer(void* arg){
   for (int i = 1; i <= 5; i++){
      sem_wait(&mutex);
      int item = buffer[out];
      printf("Consumer consumed: %d\n", item);
      out = (out + 1) % SIZE;
      sleep(2);
   }
   return NULL;
}
int main(){
   pthread_t p, c;
   sem_init(&mutex, 0, 0);
   pthread_create(&p, NULL, producer, NULL);
   pthread_create(&c, NULL, consumer, NULL);
   pthread_join(p, NULL);
   pthread_join(c, NULL);
   sem_destroy(&mutex);
```

```
    return 0;
}
```

Output:



e) Demonstrate a simple program using message queues.

Description

A message queue is a kernel-managed IPC facility that stores discrete messages until a receiving process reads them. In this program, the parent creates a System V message queue and sends a type-1 message. The child receives and displays it, after which the parent removes the queue.

Algorithm

1. Start and create a private message queue; report an error and stop if creation fails.
2. Create a child process using fork().
3. Child: receive a message of type 1, display its text and exit.
4. Parent: set the message type to 1 and its text to Hello from Parent.
5. Send the message and display a confirmation.
6. Wait for the child to finish, remove the message queue and terminate.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/wait.h>
#include <sys/msg.h>
#include <unistd.h>
struct message{
    long type;
    char text[100];
};
```

```
int main(){
    int msgid;
    struct message msg;
    msgid = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);
    if (msgid == -1){
        printf("Message queue creation failed\n");
        return 1;
    }
    if (fork() == 0){
        msgrcv(msgid, &msg, sizeof(msg.text), 1, 0);
        printf("Child received: %s\n", msg.text);
        exit(0);
    }
    else{
        msg.type = 1;
        strcpy(msg.text, "Hello from Parent");
        msgsnd(msgid, &msg, sizeof(msg.text), 0);
        printf("Parent sent: %s\n", msg.text);
        wait(NULL);
        msgctl(msgid, IPC_RMID, NULL);
    }
    return 0;
}
```

Output:

